

C Programming: Arrays and Pointers - Lecture Notes

1. The Array-Pointer Relationship

- **Rule:** The name of an array acts as a pointer to its first element.
- **Example:**

```
int arr[5] = {10, 20, 30, 40, 50};
```

- `arr` is equivalent to `&arr[0]`
- Dereferencing: `*arr` → 10
- Arithmetic: `*(arr + 1)` → 20

2. Passing Arrays to Functions

- **Problem:** Passing large arrays by value is inefficient.
- C automatically "decays" the array into a pointer.

Syntax Options:

- **Option A (Array Notation):**

```
void printArray(int arr[], int size) {  
    // ...  
}
```

- **Option B (Pointer Notation - Preferred):**

```
void printArray(int *arr, int size) {  
    // ...  
}
```

- **Why pass size?** Pointers lose size information.

3. Passing Strings to Functions

- Strings are arrays of characters ending with a null terminator `\0`
- Loop until `\0` instead of passing size:

```
void printString(char *str) {
    while (*str != '\0') {
        printf("%c", *str);
        str++;
    }
}
```

4. Function Pointers (Advanced)

- Concept: A pointer can point to a function.
- Use: Pass logic as an argument (callbacks, e.g., `qsort`)
- **Syntax:**

```
return_type (*pointer_name)(parameter_types);
```

- **Example:**

```
int add(int a, int b) {
    return a + b;
}

int main() {
    int (*funcPtr)(int, int);
    funcPtr = &add;
    int result = (*funcPtr)(10, 20);
}
```

5. Lab Activity: Array Average Calculator

Objective: Calculate sum and average using pointers.

```
#include <stdio.h>

double calculateAverage(int *arr, int size) {
    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }
    return (double)sum / size;
}
```

```
int main() {
    int marks[] = {85, 90, 78, 92, 88};
    int n = 5;

    double avg = calculateAverage(marks, n);
    printf("The average marks are: %.2f", avg);
    return 0;
}
```

Note 1 :

- **Analogy 1: Passing Arrays (The House Key)**

- Pass by value: copying the whole house → expensive
- Pass by pointer: give the address → fast and modifies the original

- **Analogy 2: Function Pointers (The Universal Remote)**

- Normal function call → fixed button
- Function pointer → programmable button (can change the function it calls)

Note 2 :

- **The sizeof Trap:** `sizeof(arr)` inside a function gives pointer size, not array size.
- **Pointer Arithmetic:** Show `str` pointing to characters and how `str++` moves the pointer.
- **Parentheses Matter:** `(*funcPtr)` is required for function pointers.